REPUBLIQUE DU SENEGAL



Un peuple-un but-une foi

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Direction de l'Enseignement Supérieur Privé

Institut Supérieur d'Informatique

ISI

Déploiement et sécurisation des communications distantes avec OpenVPN et WireGuard

Présenté par :

Mme Bintou LO

Mme Ndeye Marieme DIALLO

Sous la direction de :

Mr NDIAYE

Année académique 2025-2026

Sommaire

I. Introduction générale 2

II. Déploiement et sécurisation des communications distantes avec OpenVPN et WireGuard 3

A. Déploiement et configuration d'une solution VPN avec OpenVPN sous Ubuntu . 3

1. Definition 3

2. Configuration 3

B. Déploiement et configuration de WireGuard sous Ubuntu 16

1. Definition 16

2. Configuration 16

III. Comparaison entre OpenVPN et WireGuard 24

IV. Conclusion 25

I. Introduction générale

Dans un contexte où les utilisateurs et les collaborateurs accèdent de plus en plus aux ressources informatiques à distance, la sécurisation des communications devient un enjeu majeur pour les entreprises et les organisations. Les réseaux publics, notamment Internet, peuvent exposer les données échangées à différents risques tels que l'interception, l'écoute ou l'altération des communications.

Pour répondre à ces problématiques, les réseaux privés virtuels (VPN) permettent de créer un tunnel sécurisé entre un utilisateur distant et un réseau ou un serveur. Dans le cadre de ce travail, nous avons étudié et mis en œuvre deux solutions VPN open source : OpenVPN et WireGuard.

L'objectif est de déployer ces deux solutions sur Ubuntu afin de sécuriser les communications distantes, puis de vérifier leur bon fonctionnement à travers différents tests.

Ce travail permet également de comparer deux approches de mise en place d'un VPN sécurisé, en prenant en compte leur configuration, leur fonctionnement et leur capacité à assurer la confidentialité et la sécurité des communications.

Prérequis et environnement de travail

Pour réaliser ce travail pratique, nous avons utilisé un environnement virtualisé permettant de mettre en place les différents composants nécessaires au fonctionnement des solutions VPN.

Matériel et logiciels utilisés

- Logiciel de virtualisation : VMware Workstation
- Système d'exploitation : Ubuntu 24.04 LTS
- Solutions VPN étudiées : OpenVPN et WireGuard
- Environnement réseau : réseau virtuel configuré dans VMware
- Terminal Linux : utilisé pour l'installation et la configuration des services

- Connexion réseau : nécessaire pour l'installation des paquets et la communication entre les machines virtuelles

  L'utilisation de VMware permet de créer et d'administrer des machines virtuelles Ubuntu indépendantes, tout en reproduisant un environnement réseau adapté aux tests. Ubuntu 24.04 LTS a été choisi comme système d'exploitation pour installer et configurer successivement OpenVPN et WireGuard.

II. Déploiement et sécurisation des communications distantes avec OpenVPN et WireGuard

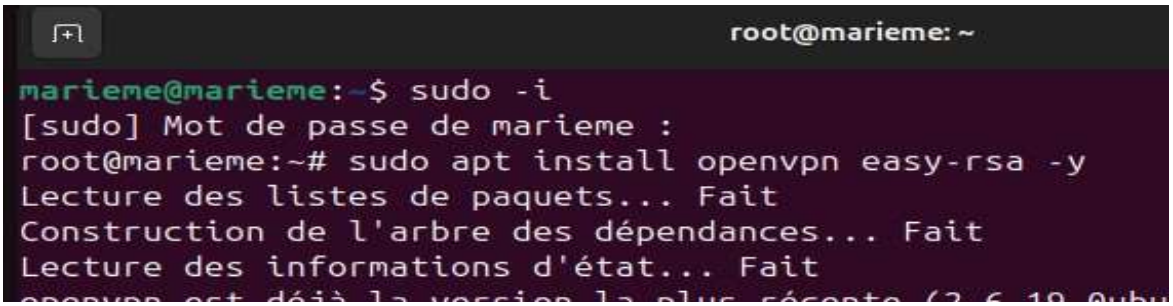
A. Déploiement et configuration d'une solution VPN avec OpenVPN sous Ubuntu

1. Definition

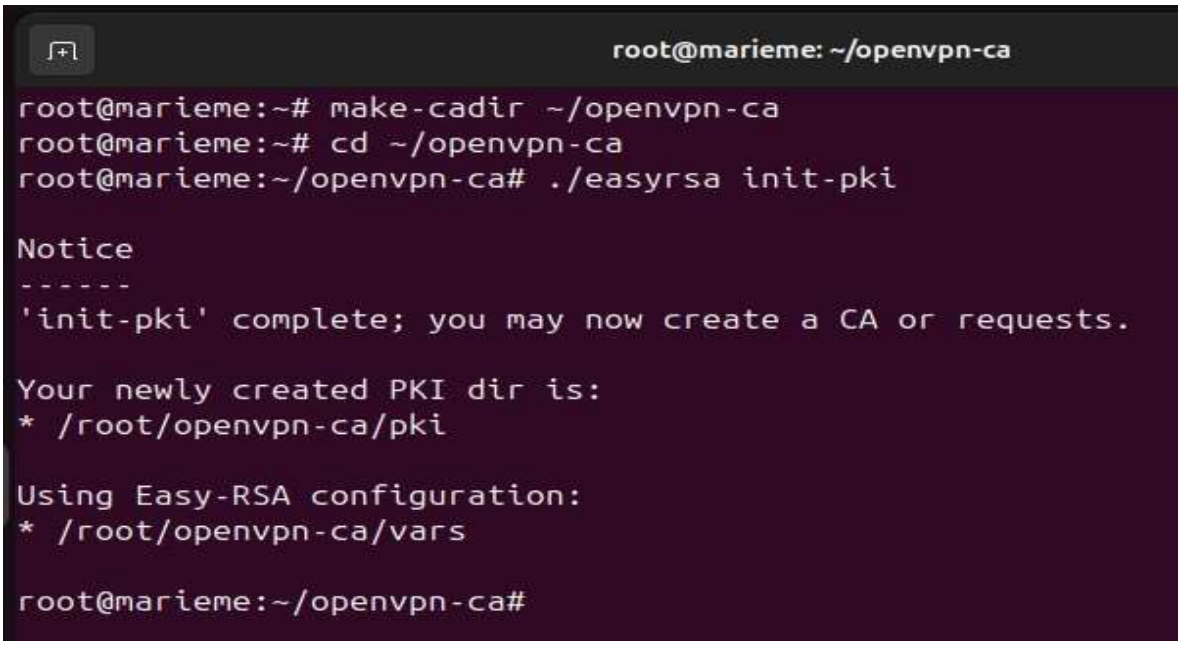
OpenVPN est une solution VPN open source permettant de sécuriser les communications entre des utilisateurs distants et un réseau privé. Il crée un tunnel VPN chiffré à travers un réseau non sécurisé, notamment Internet, afin de garantir la confidentialité et l'intégrité des données échangées. Dans le cadre de ce travail, OpenVPN a été déployé et configuré sur un environnement Ubuntu afin de mettre en place une connexion distante sécurisée. Cette partie présente les différentes étapes de l'installation, de la configuration du serveur et du client OpenVPN, ainsi que les tests permettant de vérifier le bon fonctionnement du tunnel VPN.

2. Configuration

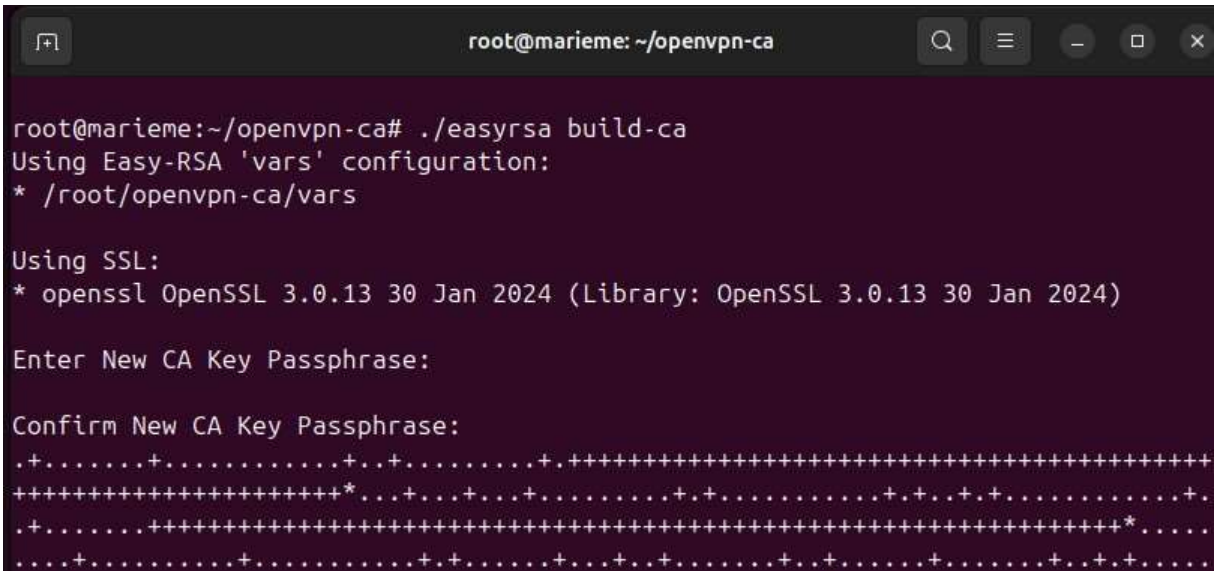
Passage en mode administrateur (root) avec la commande sudo -i, puis lancement de l'installation des paquets OpenVPN et Easy-RSA à l'aide du gestionnaire de paquets APT.



Création du répertoire de travail openvpn-ca avec make-cadir, puis initialisation d'une nouvelle structure PKI vierge à l'aide de la commande ./easyrsa init-pki



Lancement de la commande ./easyrsa build-ca pour générer le certificat racine de l'autorité de certification et définition d'un mot de passe de protection (*passphrase*).



Exécution de la commande ./easyrsa gen-req server nopass afin de créer la clé privée du serveur et la demande de signature de certificat (CSR) sans mot de passe.



Validation du nom du serveur (*Common Name*) et confirmation de la création réussie de la clé privée (server.key) ainsi que du fichier de requête (server.req).

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Common Name (eg: your user, host, or server name) [server]:

Notice
------
Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /root/openvpn-ca/pki/reqs/server.req
* key: /root/openvpn-ca/pki/private/server.key

root@marieme:~/openvpn-ca#
```

Lancement de la commande ./easyrsa sign-req server server pour demander à l'Autorité de Certification d'approuver et de signer le certificat du serveur.

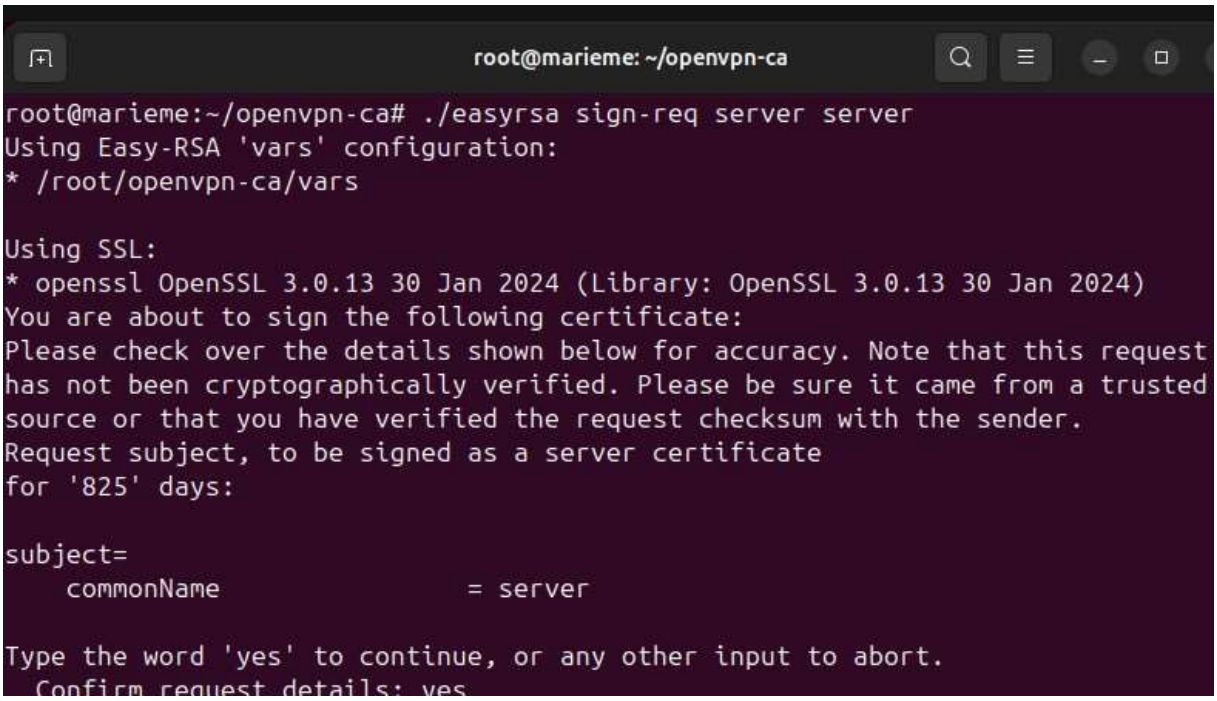
```
Using configuration from /root/openvpn-ca/pki/openssl-easyrsa.cnf
Enter pass phrase for /root/openvpn-ca/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'server'
Certificate is to be certified until Nov  9 21:20:15 2028 GMT (825 days)

Write out database with 1 new entries
Database updated

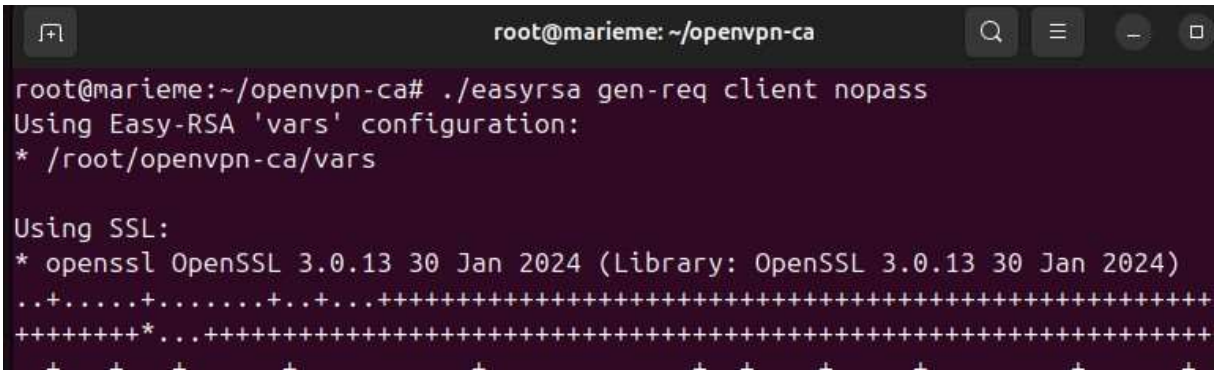
Notice
------
Certificate created at:
* /root/openvpn-ca/pki/issued/server.crt

root@marieme:~/openvpn-ca#
```

Saisie du mot de passe de l'Autorité de Certification (CA) pour signer la demande et confirmation de la création du certificat serveur valide (server.crt).



Lancement de la commande ./easyrsa gen-req client nopass afin de créer la clé privée et la demande de certificat pour un utilisateur ou appareil client.



Exécution de la commande ./easyrsa sign-req client client pour initier la signature du certificat client par l'Autorité de Certification (CA).

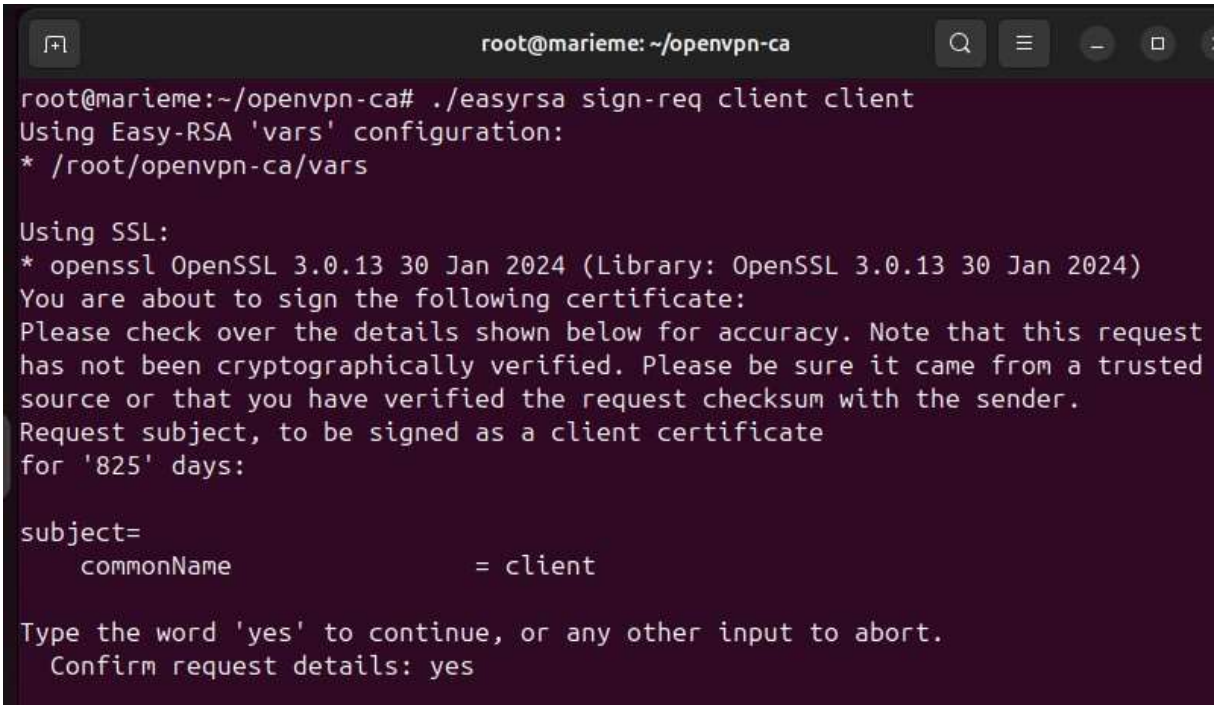
```
Using configuration from /root/openvpn-ca/pki/openssl-easyrsa.cnf
Enter pass phrase for /root/openvpn-ca/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'client'
Certificate is to be certified until Nov  9 21:31:56 2028 GMT (825 days)

Write out database with 1 new entries
Database updated

Notice
------
Certificate created at:
* /root/openvpn-ca/pki/issued/client.crt

root@marieme:~/openvpn-ca#
```

Authentification par le mot de passe de la CA pour signer la requête et génération finale du fichier de certificat du client (client.crt).



Copie du certificat de l'autorité de certification (ca.crt), de la clé privée du serveur (server.key) et du certificat signé du serveur (server.crt) vers le répertoire de configuration principal /etc/openvpn/server/.

```
root@marieme:~/openvpn-ca# cp pki/ca.crt /etc/openvpn/server/
root@marieme:~/openvpn-ca# cp pki/private/server.key /etc/openvpn/server/
root@marieme:~/openvpn-ca# cp pki/issued/server.crt /etc/openvpn/server/
```

Confirmation de la fin du processus avec la création réussie du fichier de paramètres Diffie-Hellman à l'emplacement /root/openvpn-ca/pki/dh.pem.

```
++*++++*++*++++*++++*++++*++++*++++*++++*++++*++++*++++*++++*++++*++++*++++*++
DH parameters appear to be ok.

Notice
------

DH parameters of size 2048 created at:
* /root/openvpn-ca/pki/dh.pem
```

Lancement de la commande Easy-RSA ./easyrsa gen-dh pour débuter la génération des paramètres Diffie-Hellman (2048 bits), indispensables pour assurer l'échange sécurisé de clés de chiffrement.

```
root@marieme:~/openvpn-ca# ./easyrsa gen-dh
Using Easy-RSA 'vars' configuration:
* /root/openvpn-ca/vars

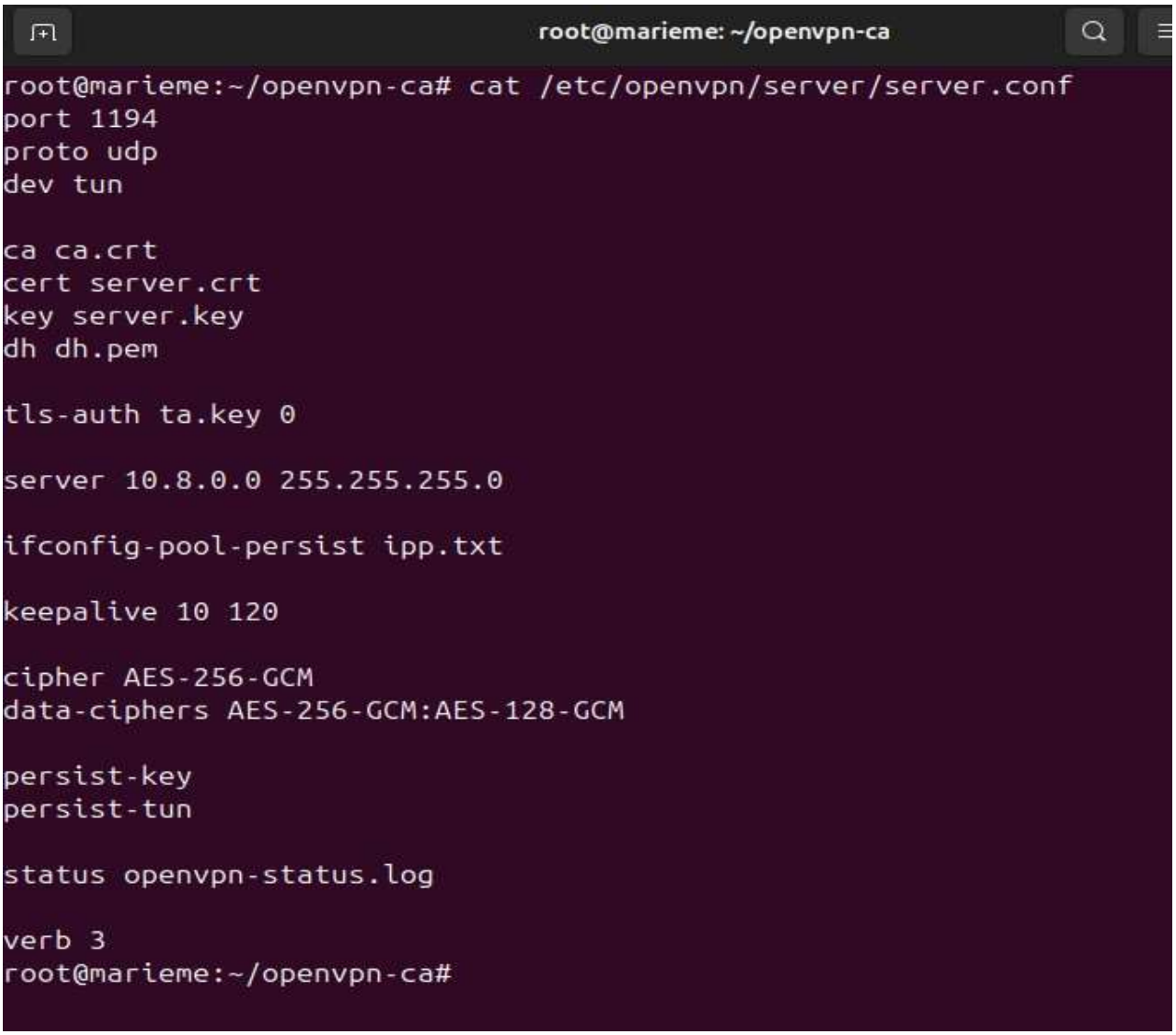
Using SSL:
* openssl OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
Generating DH parameters, 2048 bit long safe prime
.....................................................................+.....
.....................+.............+.....................................
.....................................................+.....................
```

Génération d'une clé secrète TLS-Auth (ta.key) via OpenVPN pour protéger le serveur contre les attaques de type DoS et le balayage de ports, suivie de la copie des fichiers dh.pem et ta.key dans le dossier /etc/openvpn/server/.

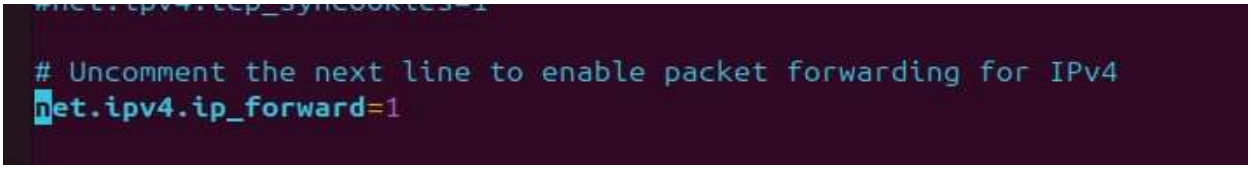
```
root@marieme: ~/openvpn-ca
root@marieme:~/openvpn-ca# openvpn --genkey secret ta.key
root@marieme:~/openvpn-ca# cp pki/dh.pem /etc/openvpn/server/
root@marieme:~/openvpn-ca# cp ta.key /etc/openvpn/server/
root@marieme:~/openvpn-ca#
```

Affichage du contenu du fichier de configuration du serveur (server.conf), définissant le port d'écoute (1194 UDP), le mode TUN, l'adressage IP du réseau

virtuel (10.8.0.0/24), les clés/certificats utilisés, les algorithmes de chiffrement (AES-256-GCM) et les options de maintien de connexion.



Modification dans le fichier de configuration du noyau pour activer l'option net.ipv4.ip_forward=1, indispensable pour autoriser le serveur à rout er et transférer les paquets réseau entre les clients VPN et le réseau externe.



Commande d'ouverture du fichier /etc/sysctl.conf avec l'éditeur de texte Vim pour éditer la configuration des paramètres du noyau Linux.

```
root@marieme:~/openvpn-ca# vim /etc/sysctl.conf
```

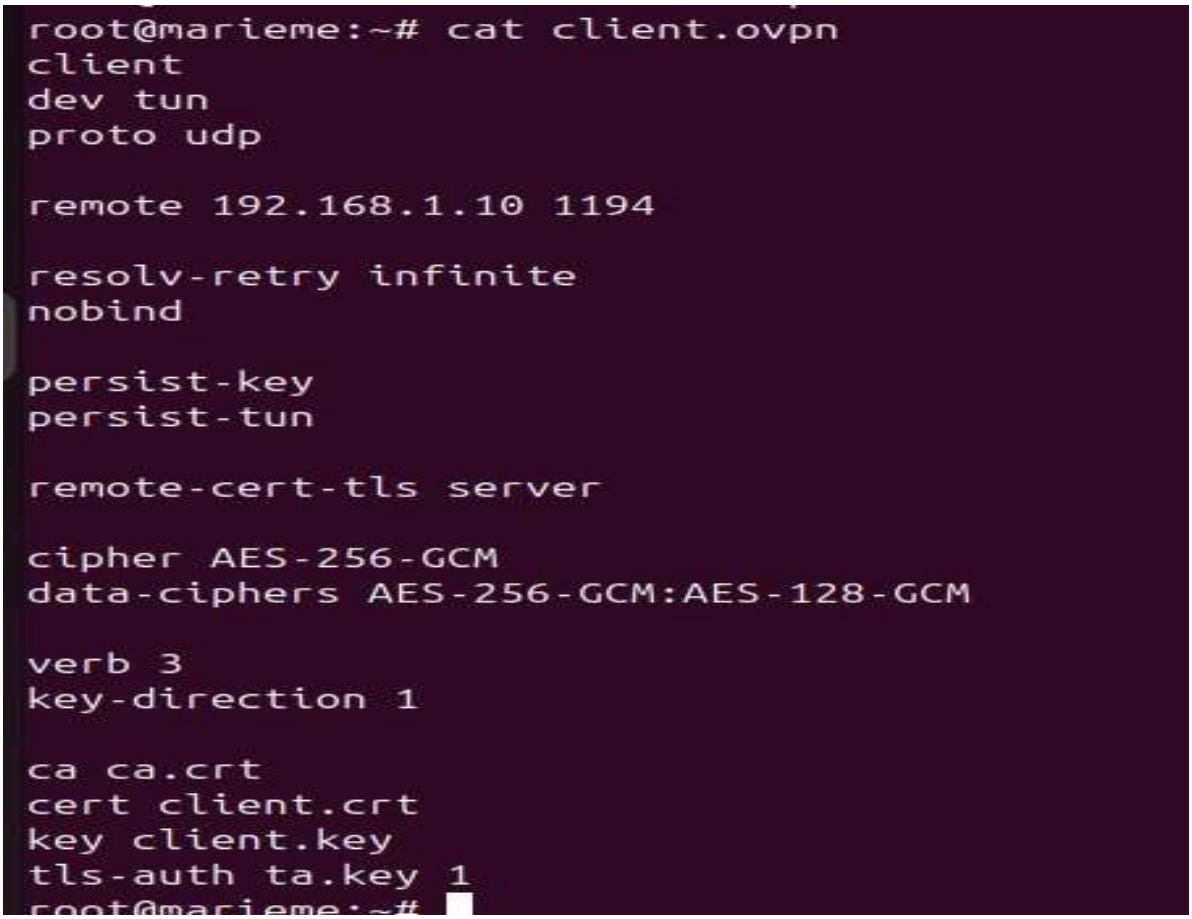
Exécution de sysctl -p pour appliquer immédiatement les modifications système sans redémarrage, suivi de l'ajout d'une règle dans le pare-feu UFW (ufw allow 1194/udp) pour autoriser le flux VPN entrant.

```
root@marieme:~/openvpn-ca# sysctl -p
net.ipv4.ip_forward = 1
root@marieme:~/openvpn-ca# ufw allow 1194/udp
Les règles ont été mises à jour
Les règles ont été mises à jour (IPv6)
root@marieme:~/openvpn-ca#
```

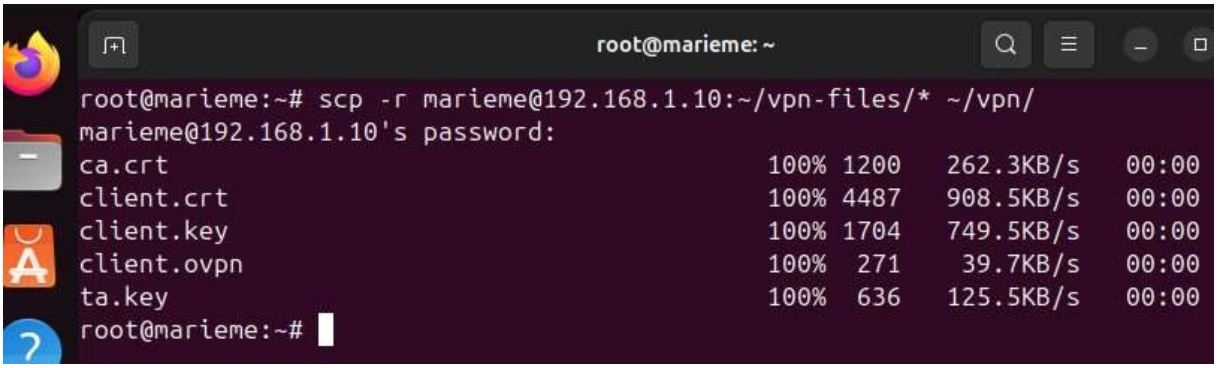
Activation du service au démarrage (systemctl enable), démarrage de l'instance du serveur OpenVPN (systemctl start) et vérification de son état de fonctionnement (systemctl status), confirmant que le service est actif et correctement initialisé.

```
root@marieme:~/openvpn-ca# systemctl enable openvpn-server@server
Created symlink /etc/systemd/system/multi-user.target.wants/openvpn-server@server
.service → /usr/lib/systemd/system/openvpn-server@.service.
root@marieme:~/openvpn-ca# systemctl start openvpn-server@server
root@marieme:~/openvpn-ca# systemctl status openvpn-server@server
● openvpn-server@server.service - OpenVPN service for server
     Loaded: loaded (/usr/lib/systemd/system/openvpn-server@.service; enabled; p>
     Active: active (running) since Fri 2026-08-07 21:56:05 GMT; 14s ago
       Docs: man:openvpn(8)
             https://openvpn.net/community-resources/reference-manual-for-openvp>
             https://community.openvpn.net/openvpn/wiki/HOWTO
   Main PID: 4803 (openvpn)
     Status: "Initialization Sequence Completed"
      Tasks: 1 (limit: 3191)
     Memory: 1.4M (peak: 1.6M)
        CPU: 12ms
     CGroup: /system.slice/system-openvpn\x2dserver.slice/openvpn-server@server.>
             └─4803 /usr/sbin/openvpn --status /run/openvpn-server/status-server>
```

Affichage du fichier de profil client (client.ovpn) précisant les paramètres de connexion à destination du serveur (192.168.1.10 sur le port UDP 1194), les méthodes de chiffrement, ainsi que l'emplacement des certificats et clés nécessaires au client.


root@marieme:~# cat client.ovpn
client
dev tun
proto udp
remote 192.168.1.10 1194
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
cipher AES-256-GCM
data-ciphers AES-256-GCM:AES-128-GCM
verb 3
key-direction 1
ca ca.crt
cert client.crt
key client.key
tls-auth ta.key 1

Copie sécurisée à distance via la commande scp -r de l'ensemble des fichiers nécessaires au client OpenVPN (ca.crt, client.crt, client.key, client.ovpn, ta.key) depuis la machine serveur (192.168.1.10) vers le répertoire local ~/vpn/.



Navigation vers le dossier local contenant les fichiers téléchargés (cd /root/vpn), puis lancement du client OpenVPN en ligne de commande à l'aide de la configuration spécifiée (openvpn --config client.ovpn).

```
root@marieme:~# cd /root/vpn
openvpn --config client.ovpn
```

Affichage du journal d'initialisation OpenVPN côté client confirmant la réception des routes pushing, le paramétrage de l'interface virtuelle tun0 avec l'adresse IP 10.8.0.6 et la finalisation de l'établissement du tunnel sécurisé (*Initialization Sequence Completed*).

```
o trusted
2026-08-09 18:52:22 PUSH: Received control message: 'PUSH_REPLY,route 10.8.0.1,t
opology net30,ping 10,ping-restart 120,ifconfig 10.8.0.6 10.8.0.5,peer-id 0,ciph
er AES-256-GCM,protocol-flags cc-exit tls-ekm dyn-tls-crypt,tun-mtu 1500'
2026-08-09 18:52:22 OPTIONS IMPORT: --ifconfig/up options modified
2026-08-09 18:52:22 OPTIONS IMPORT: route options modified
2026-08-09 18:52:22 OPTIONS IMPORT: tun-mtu set to 1500
2026-08-09 18:52:22 net_route_v4_best_gw query: dst 0.0.0.0
2026-08-09 18:52:22 net_route_v4_best_gw result: via 192.168.231.2 dev ens34
2026-08-09 18:52:22 ROUTE_GATEWAY 192.168.231.2/255.255.255.0 IFACE=ens34 HWADDR
=00:0c:29:97:16:2a
2026-08-09 18:52:22 TUN/TAP device tun0 opened
2026-08-09 18:52:22 net_iface_mtu_set: mtu 1500 for tun0
2026-08-09 18:52:22 net_iface_up: set tun0 up
2026-08-09 18:52:22 net_addr_ptp_v4_add: 10.8.0.6 peer 10.8.0.5 dev tun0
2026-08-09 18:52:22 net_route_v4_add: 10.8.0.1/32 via 10.8.0.5 dev [NULL] table
0 metric -1
2026-08-09 18:52:22 Initialization Sequence Completed
2026-08-09 18:52:22 Data Channel: cipher 'AES-256-GCM', peer-id: 0
2026-08-09 18:52:22 Timers: ping 10, ping-restart 120
2026-08-09 18:52:22 Protocol options: protocol-flags cc-exit tls-ekm dyn-tls-cry
pt
```

a. Tests de fonctionnement

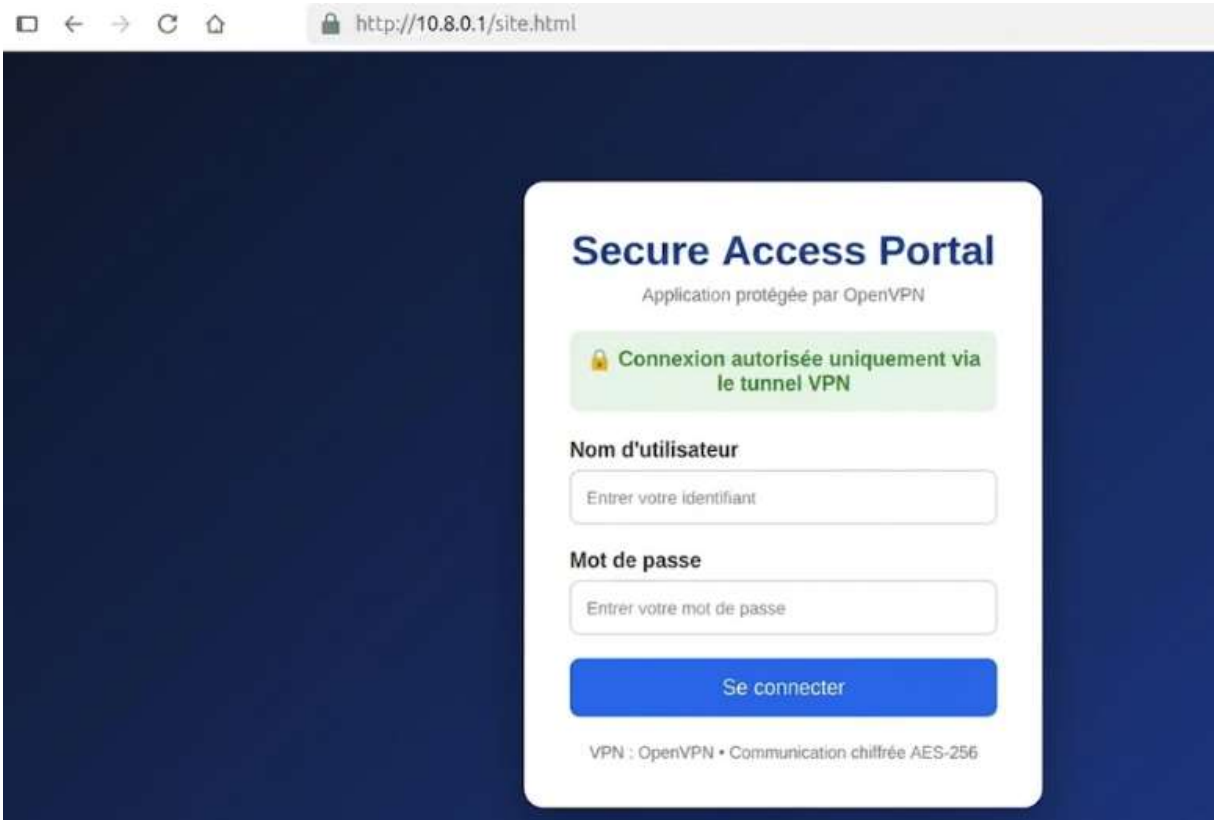
Test de connectivité ICMP (ping 10.8.0.1) depuis le client vers l'adresse IP virtuelle du serveur OpenVPN, affichant 0% de perte de paquets et validant ainsi le bon fonctionnement de la communication au sein du tunnel VPN.

```
root@marieme:~# ping 10.8.0.1
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data.
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=1.66 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=2.08 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=1.84 ms
64 bytes from 10.8.0.1: icmp_seq=4 ttl=64 time=2.43 ms
64 bytes from 10.8.0.1: icmp_seq=5 ttl=64 time=1.87 ms
^C
--- 10.8.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 1.656/1.975/2.434/0.265 ms
```

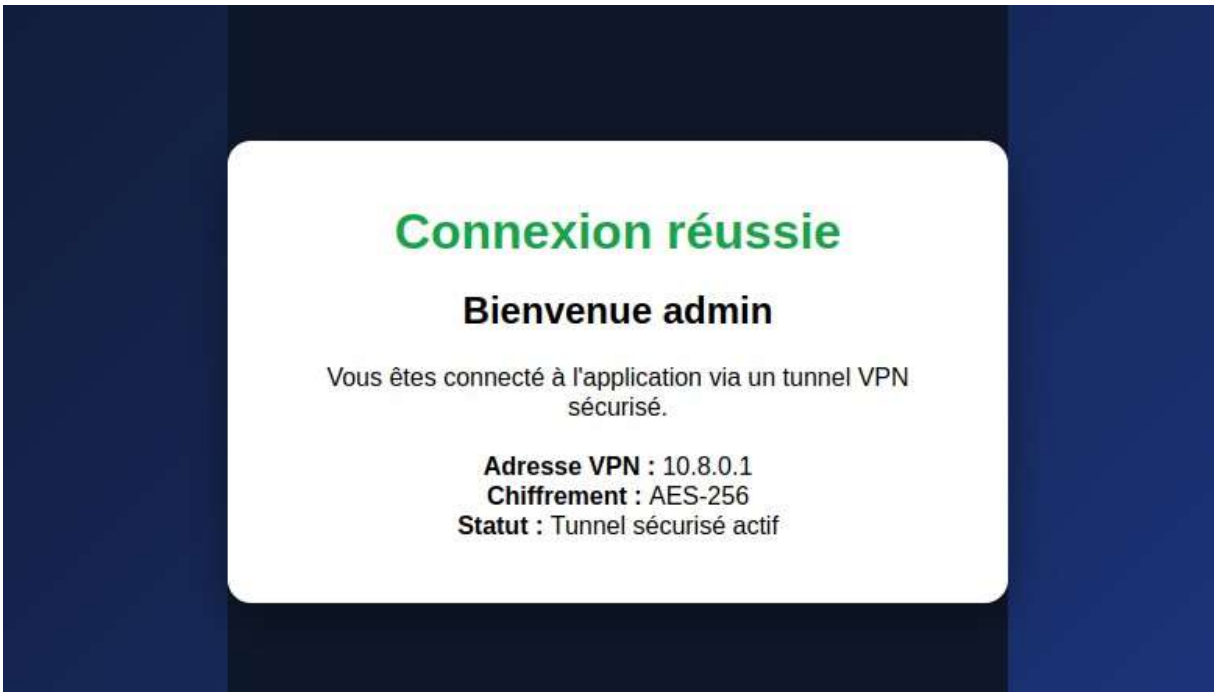
Vérification des interfaces réseau locales montrant l'état de l'interface tunnel tun0, configurée avec l'adresse IP point-à-point 10.8.0.1 liée à l'extrémité du serveur VPN.

```
4: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
 UNKNOWN group default qlen 500
    link/none
    inet 10.8.0.1 peer 10.8.0.2/32 scope global tun0
       valid_lft forever preferred_lft forever
    inet6 fe80::22e2:9482:8191:b60e/64 scope link stable-privacy
       valid_lft forever preferred_lft forever
root@marieme:~#
```

Accès depuis le navigateur web à la page d'authentification du portail d'accès sécurisé (*Secure Access Portal*) hébergé sur le serveur VPN à l'adresse IP privée 10.8.0.1.



Affichage de la page de confirmation de connexion réussie pour l'utilisateur administrateur, validant l'accès sécurisé à l'application web via le tunnel VPN actif en chiffrement AES-256.



B. Déploiement et configuration de WireGuard sous Ubuntu

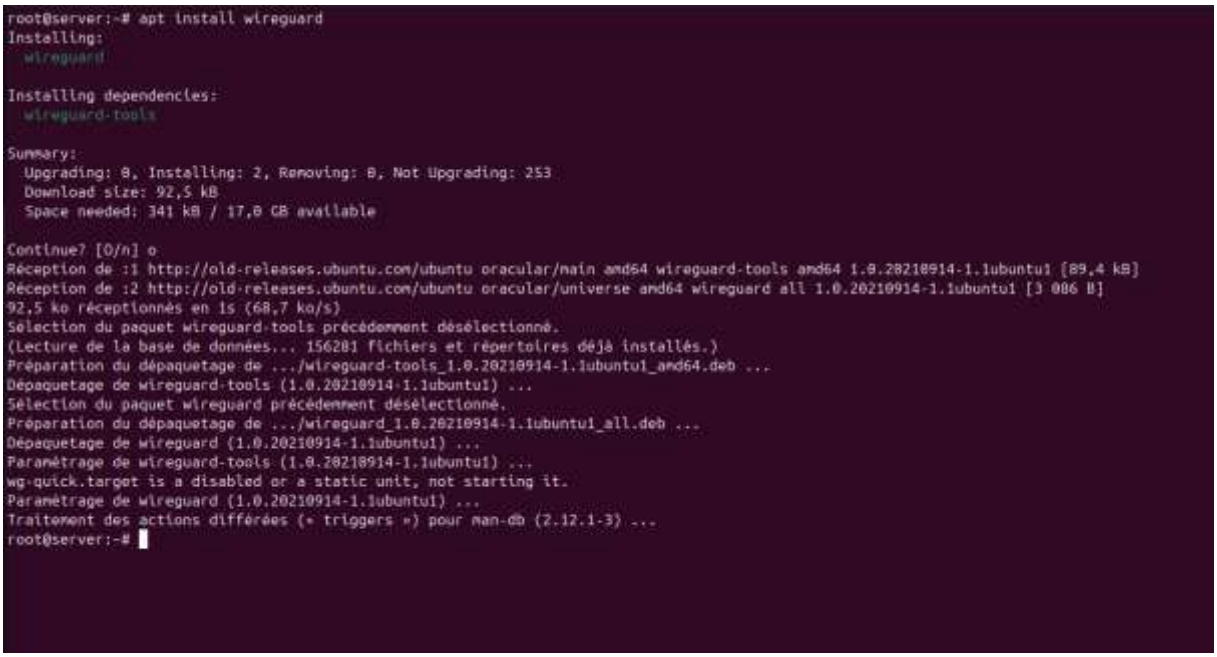
1. Definition

Pour sécuriser la communication entre nos serveurs, nous avons implémenté le protocole WireGuard. Ce protocole VPN moderne est reconnu pour sa simplicité, sa rapidité et sa sécurité cryptographique forte. La configuration établit un tunnel de communication chiffré basé sur l'échange de clés publiques, garantissant ainsi l'intégrité et la confidentialité des échanges entre les machines.

2. Configuration

a. Configuration du serveur

L'implémentation du protocole WireGuard sur les serveurs a d'abord nécessité l'installation du paquet via le gestionnaire apt :



Pour garantir la sécurité cryptographique du tunnel, nous avons procédé à la génération des paires de clés publiques et privées pour le serveur en utilisant la commande suivante :

```
root@server:~# wg genkey | tee privatekey | wg pubkey > publickey
```



Nous éditons le fichier de configuration WireGuard, /etc/wireguard/wg0.conf à l'aide de l'éditeur vim : vim /etc/wireguard/wg0.conf

Le contenu du fichier /etc/wireguard/wg0.conf détaille les paramètres de l'interface WireGuard. On y définit l'adresse IP interne du tunnel (10.0.0.1/8), le port d'écoute (51820), ainsi que les règles PostUp et PostDown utilisant iptables. Ces règles sont cruciales pour autoriser le trafic VPN à travers le pare-feu du serveur



On active le service



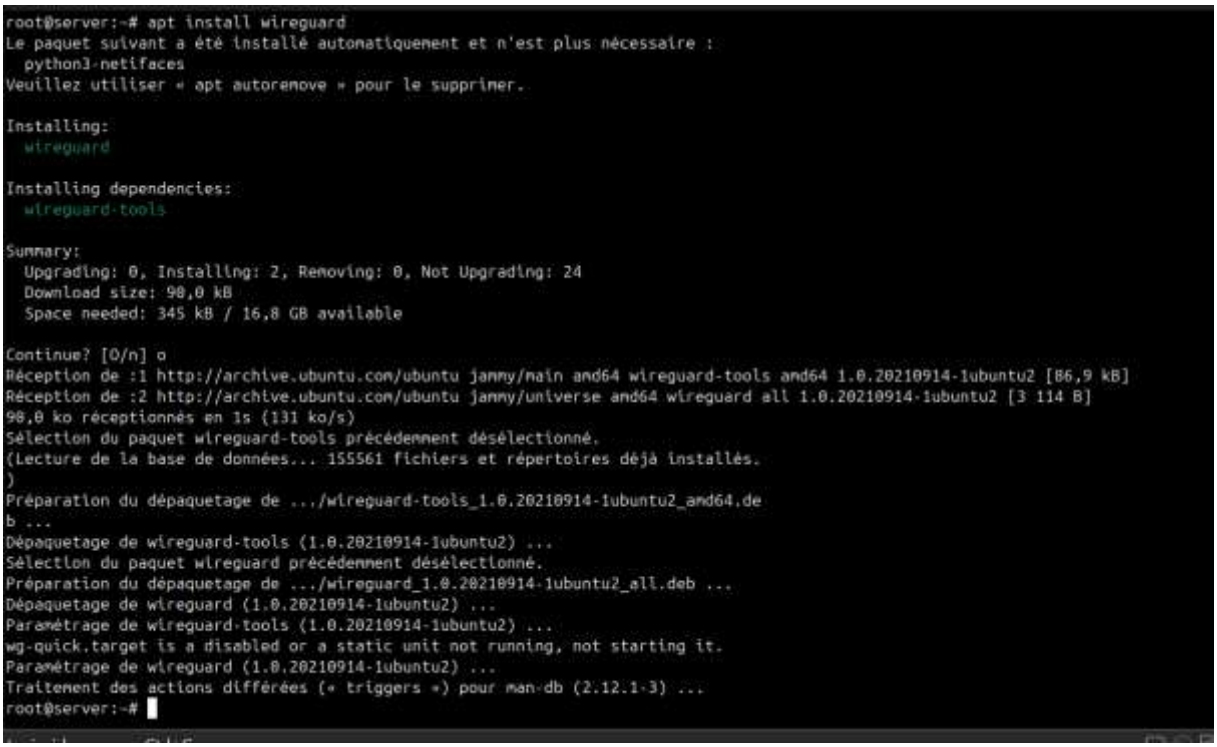
Après avoir activé le service, la commande sudo wg confirme que l'interface wg0 est active. On voit que la clé publique du serveur est générée et qu'elle écoute correctement sur le port 51820, attestant que le tunnel est prêt à recevoir des connexions de pairs.



b. Configuration du client

On a également exécuté cette même commande d'installation (apt install wireguard -y) sur le serveur client destiné à se connecter au tunnel WireGuard.

Ceci est nécessaire pour que le client puisse générer ses propres paires de clés et utiliser le protocole.



Le serveur client doit également générer sa propre paire de clés. La commande est répétée pour créer la clé privée et la clé publique du client :



Ensuite nous éditons de même le fichier de configuration /etc/wireguard/wg0.conf sur le serveur client. Le contenu du fichier de configuration client /etc/wireguard/wg0.conf reflète les paramètres essentiels du tunnel. Nous y voyons l'adresse IP interne du client (10.0.0.2/8) et dans la section [Peer], les informations du serveur : sa clé publique (PublicKey) et son point de terminaison (Endpoint).

```
[Interface]
PrivateKey = cPAvxDR5AY28p2+NGlk1HlZkeronD+kOlZ4KavBsH3Y=
Address = 10.0.0.2/8
SaveConfig = true

[Peer]
PublicKey = 7J3KcHU5zwaldMyjhRkHnQIzsUWgGbP7+ibmsVjo6B8=
Endpoint = 192.168.100.1:51820
AllowedIPs = 0.0.0.0/0
```

On active l’interface

```
root@server:~# wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.2/8 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] wg set wg0 fwmark 51820
[#] ip -4 route add 0.0.0.0/0 dev wg0 table 51820
[#] ip -4 rule add not fwmark 51820 table 51820
[#] ip -4 rule add table main suppress_prefixlength 0
[#] sysctl -q net.ipv4.conf.all.src_valid_mark=1
[#] nft -f /dev/fd/63
root@server:~#
```

Nous utilisons la commande wg set pour ajouter dynamiquement le pair client au serveur. Cette commande spécifie la clé publique du client et l'adresse IP interne qui lui est autorisée mettant à jour l'interface wg0 du serveur sans nécessiter de redémarrage du service.

```
root@server:~# wg set wg0 peer Vc5XMjlDl4/6PWTebn/iAP1tb5gNu7zjmY5EhOAmJTk= allowed-ips 10.0.0.2/32
root@server:~#
```

Cette option envoie un paquet de maintien de connexion toutes les 30 secondes pour conserver le tunnel actif, notamment si le client se trouve derrière un NAT ou un pare-feu.

```
[Interface]
Address = 10.0.0.2/8
SaveConfig = true
ListenPort = 57980
FwMark = 0xca6c
PrivateKey = cPAvxDR5AY28p2+NGlk1HlZkeronD+kOlZ4KavBsH3Y=

[Peer]
PublicKey = 7J3KcHU5zwaldMyjhRkHnQIzsUWgGbP7+lbmsVjo6B8=
AllowedIPs = 0.0.0.0/0
Endpoint = 192.168.100.1:51820
PersistentKeepalive = 30
```

Pour que le serveur WireGuard puisse correctement acheminer le trafic de ses clients vers le réseau externe (grâce aux règles NAT/Masquerade définies en PostUp, On active la fonctionnalité de routage IP Forwarding La commande cat /proc/sys/net/ipv4/ip_forward vérifie cet état, et la valeur 1 confirme que le routage des paquets est bien actif sur le serveur.

```
root@server:~
root@server:~# cat /proc/sys/net/ipv4/ip_forward
1
root@server:~#
```

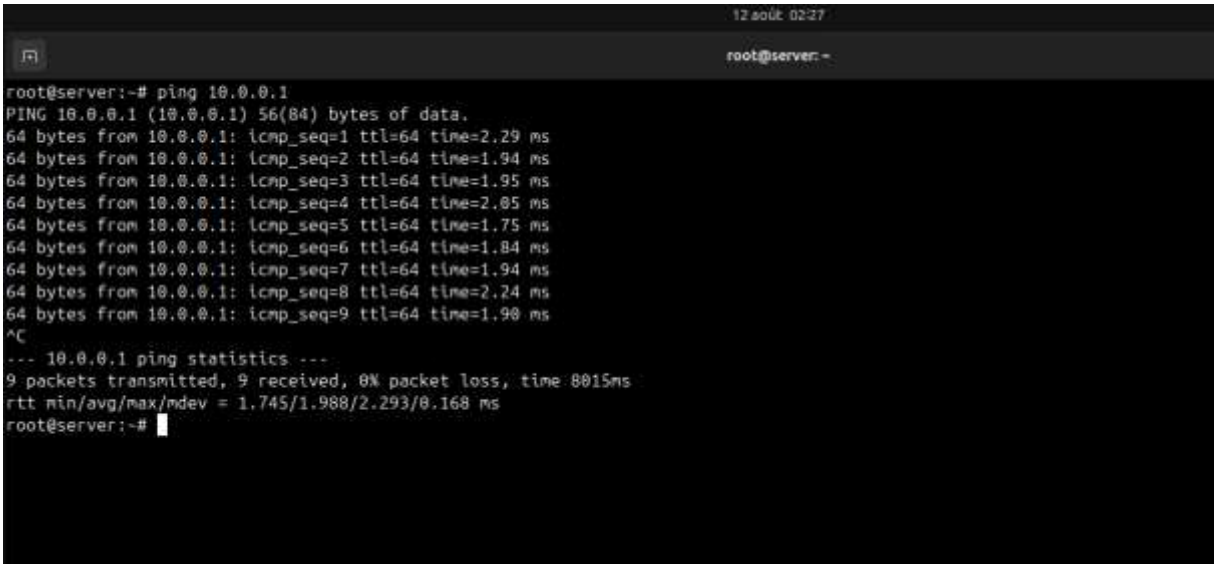
c. *Tests de fonctionnement*

L'exécution de la commande wg confirme le statut opérationnel de la connexion : la ligne latest handshake indique une poignée de main récente, et les compteurs transfer montrent que le trafic est correctement acheminé à travers le tunnel chiffré

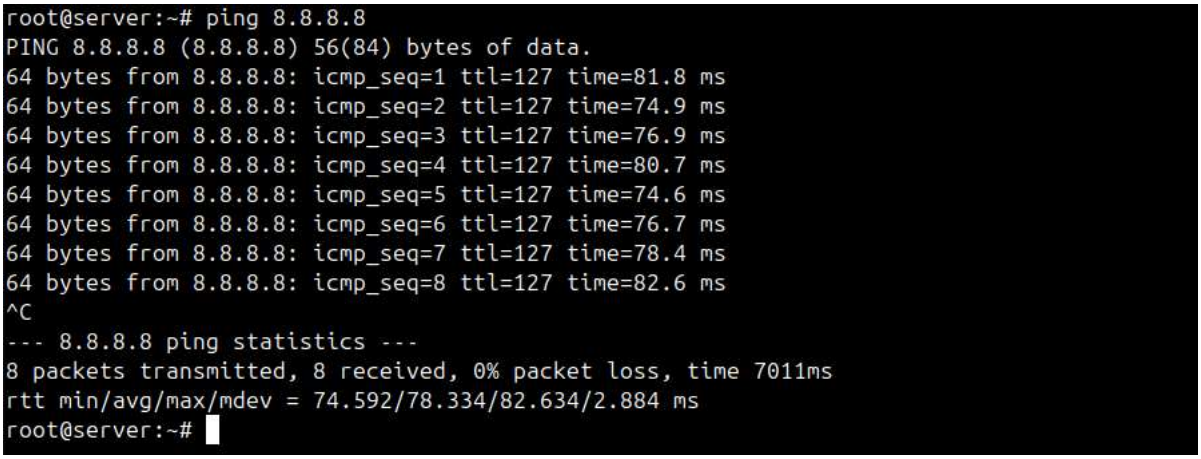
```
root@server:~# wg
interface: wg0
  public key: Vc5XMjiDl4/6PWTebn/iAP1tb5gNu7zjnY5EhOAmJTk=
  private key: (hidden)
  listening port: 57980
  fwmark: 0xca6c

peer: 7J3KcHU5zwaldMyjhRkHnQIzsUWgGbP7+lbmsVjo6B8=
  endpoint: 192.168.100.1:51820
  allowed ips: 0.0.0.0/0
  latest handshake: 2 seconds ago
  transfer: 92 B received, 180 B sent
  persistent keepalive: every 30 seconds
root@server:~#
```

Finalement, on vérifie la connectivité réseau du tunnel en exécutant une commande ping depuis le client vers l'adresse IP interne du serveur WireGuard (10.0.0.1). Le succès de cette commande confirme que la communication chiffrée est pleinement opérationnelle entre le client et le serveur.



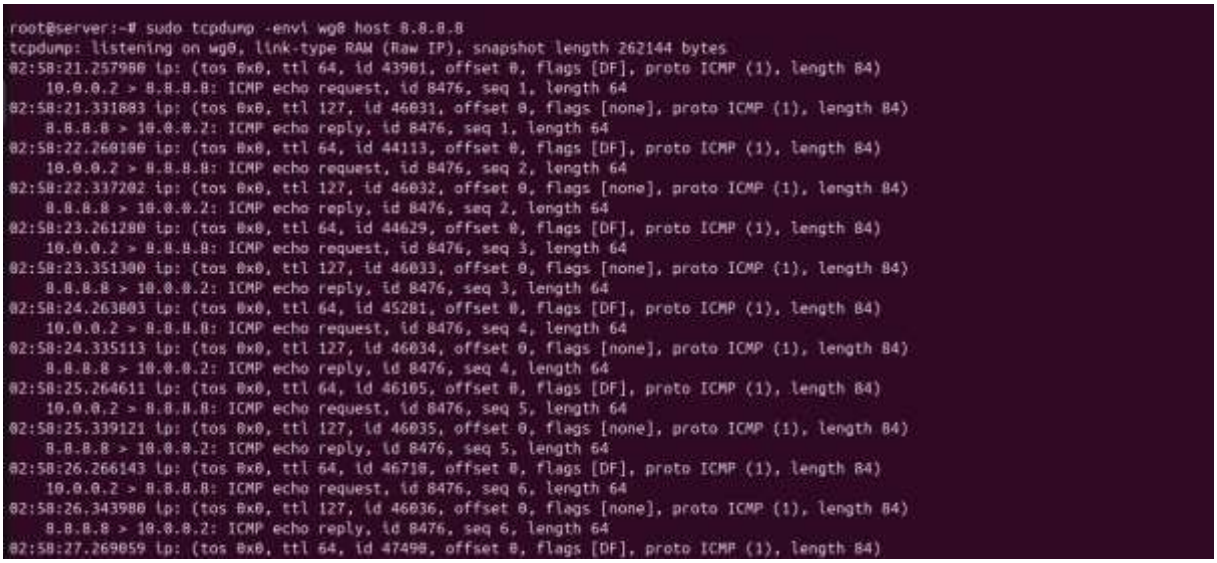
On fait un ping vers google.com



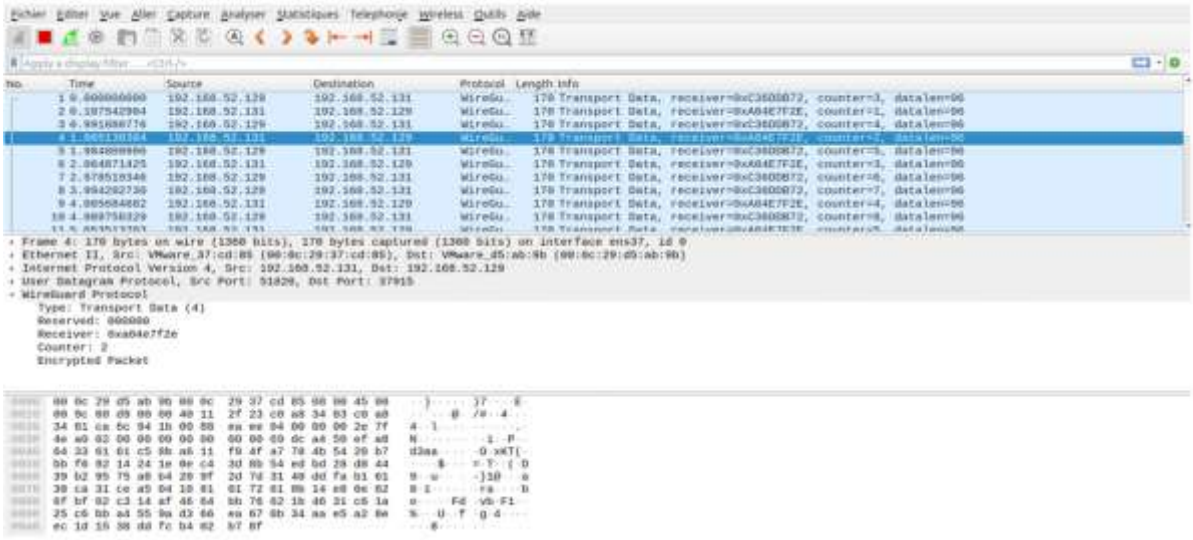
- 10.0.0.2 > 8.8.8.8: ICMP echo request : Le client envoie sa requête ICMP à travers le tunnel wg0.
- 8.8.8.8 > 10.0.0.2: ICMP echo reply : Google répond et le serveur renvoie la réponse au client à travers le tunnel.

Le routage, le tunnel chiffré WireGuard et le NAT (MASQUERADE) sur le serveur fonctionnent sans aucun problème.

Pour la validation finale du routage vers l'extérieur, on utilise la commande tcpdump sur l'interface wg0 du serveur pour inspecter le trafic. Le résultat est sans appel : on voit les paquets ICMP (ping) du client (10.0.0.2) quitter le tunnel pour atteindre une adresse externe (8.8.8.8) et revenir. Ceci confirme que le routage, y compris la règle NAT (PostUp), est pleinement fonctionnel et que le client peut accéder à Internet via le tunnel chiffré.



La preuve finale de la sécurité du tunnel WireGuard est apportée par une capture Wireshark réalisée sur l'interface externe du serveur. On confirme que le trafic circulant entre le serveur et le client est bien encapsulé sous le protocole WireGuard et, surtout, qu'il est étiqueté « Encrypted Packet ». Ceci valide que le chiffrement est effectif, assurant la confidentialité totale des données en transit.



III. Comparaison entre OpenVPN et WireGuard

Après avoir déployé et configuré OpenVPN et WireGuard sur Ubuntu, il est intéressant de comparer ces deux solutions VPN afin de mettre en évidence leurs principales différences. Cette comparaison porte notamment sur leur architecture, leur sécurité, leurs performances, leur facilité de configuration ainsi que leur consommation des ressources. Elle permet également d'identifier les avantages et les limites de chaque solution et de déterminer celle qui peut être la plus adaptée selon les besoins d'une infrastructure réseau.

Critère	OpenVPN	WireGuard
Architecture	Architecture basée sur SSL/TLS, relativement complète et configurable.	Architecture moderne, simple et légère.
Sécurité	Niveau de sécurité élevé grâce à TLS et différents mécanismes cryptographiques	Utilise des algorithmes cryptographiques modernes avec une conception minimaliste.
Performances	Bonnes performances, mais une architecture plus lourde peut entraîner davantage de surcharge.	Très bonnes performances grâce à sa conception légère.
Configuration	Configuration relativement complexe avec plusieurs fichiers et paramètres.	Configuration simple et concise.
Consommation des ressources	Consommation généralement plus importante.	Faible consommation de ressources.

Facilité de déploiement	Déploiement nécessitant plusieurs étapes de configuration.	Déploiement généralement plus rapide et plus simple.
Maintenance	Gestion pouvant être plus complexe, notamment avec les certificats.	Maintenance simplifiée grâce à une configuration concise.
Flexibilité	Très flexible et adapté à de nombreux scénarios réseau.	Simple et efficace pour de nombreux besoins de connexion distante.
Compatibilité	Compatible avec de nombreuses plateformes et largement utilisé.	Compatible avec les principales plateformes modernes.
Cas d'utilisation	Adapté aux infrastructures nécessitant une configuration VPN avancée.	Adapté aux connexions VPN modernes nécessitant simplicité et performances.

IV. Conclusion

La mise en œuvre d'OpenVPN et de WireGuard sous Ubuntu nous a permis de comprendre les principales étapes nécessaires au déploiement d'une solution VPN destinée à sécuriser les communications distantes. Les deux solutions permettent de créer un tunnel sécurisé entre les différents équipements et de protéger les données échangées sur un réseau potentiellement non sécurisé.

À travers les configurations et les tests réalisés, nous avons pu mettre en pratique les notions de chiffrement, de tunnel VPN, d'adressage IP et de communication sécurisée. Ce travail montre ainsi l'importance des VPN dans la protection des communications distantes et constitue une base pour comparer les caractéristiques d'OpenVPN et de WireGuard dans différents environnements réseau.